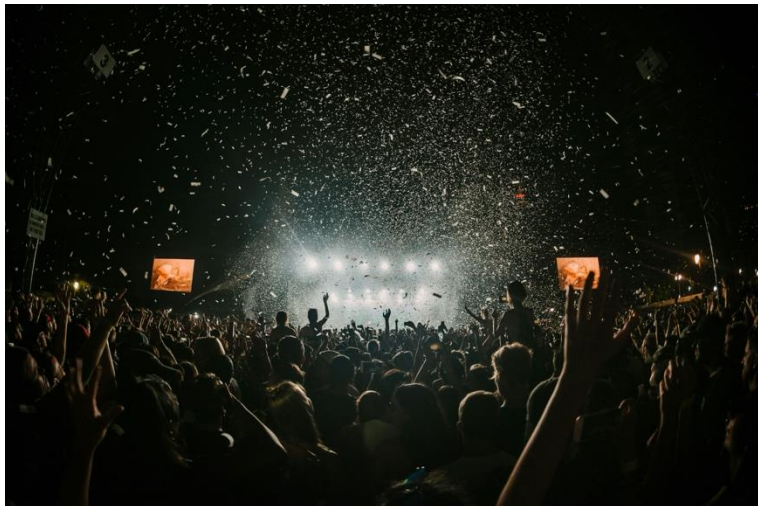




WELCOME BACK TO THE ADVENTURE!!!

CS 6650 *Scalable* Distributed Systems
(while maintaining some sanity?! 😊)



Last week and TODAY!

- Homework readings!
 - How did it **GO**?! (hahahaha!)... what were your obstacles?!
 - What did you learn?
 - How are the interviews?!
- This week!
 - Super cool paper: MapReduce (take a look around, they were ROASTED!)
 - What's really going on here?!
 - How did it all start?!

Last time: What are the REAL challenges?!

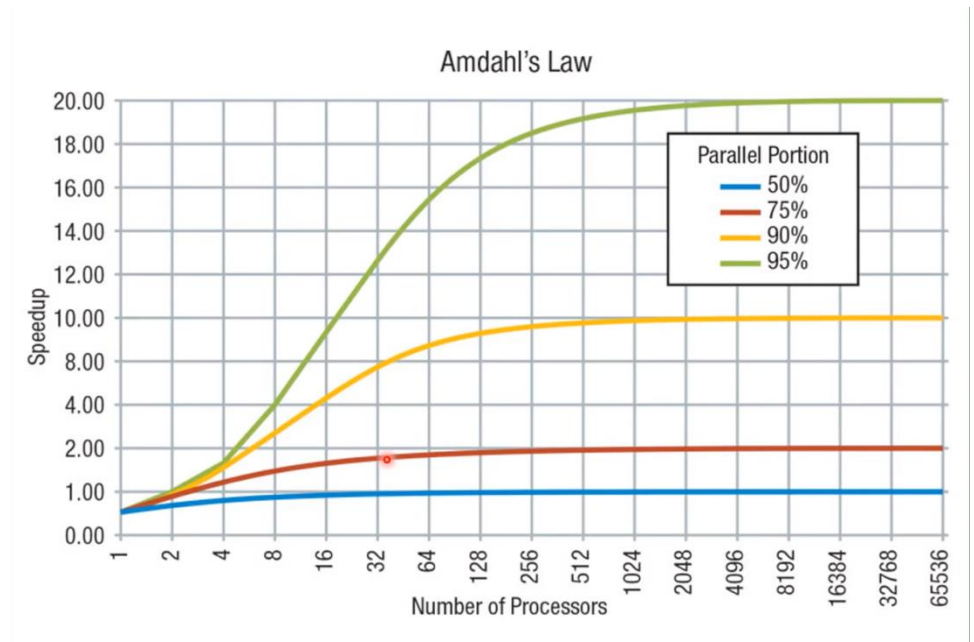
- You want to get REAL experience building these systems
 - But you also want to see the *forest* for the trees!!!
 - One way to think of this...
 - What happens when you work in large groups?!
 - Things go wrong ALLLLLLL the time!
 - How are distributed systems DIFFERENT from *working in groups*?
- Check this out when you can!
 - <https://thesecretlivesofdata.com/raft/>

What is SCALABILITY?

- The ability of a system to **increase** or **decrease** capacity in response to changing demands while continuing to satisfy service level agreements for latency and availability
- “**Elastic Compute**”

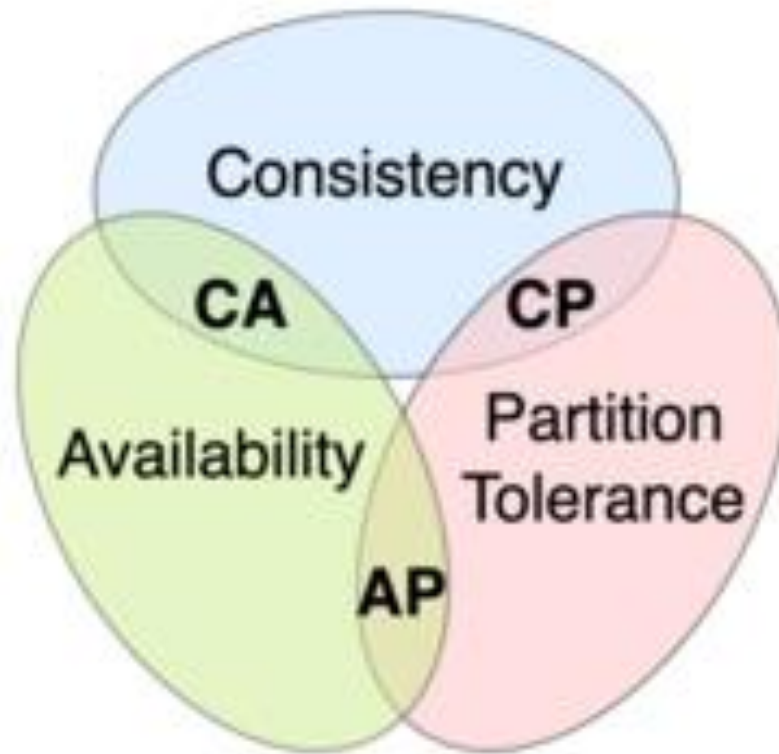
Can we just throw hardware at it?

- Yes and no!
- We want to “scale”
 - With each additional machine, we want to be able to increase our capacity!
 - BUT there is Amdahl’s Law (wikipedia here!)
 - At some point we level out
 - Not getting faster because some things have to be done in sequence, one at a time
 - Example, a write to a database
 - “Serial bottlenecks”!
- You will also love the “CAP theorem”



Wikipedia for now!

- The CAP, like a cap on what we can REALLY do!!!
- There are important fundamental tradeoffs to consider!!!



SOME problems are easily parallelizable

- MapReduce (Please share your thoughts on Piazza!)
- *EMBARRISINGLY* Parallelizable! 😊
- Some serialized behaviour but does well!
- Can be a little tricky to get right...
 - Data Skew
 - Breaking the data set down into “chunks”
 - Not every chunk takes the same amount of time to execute

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

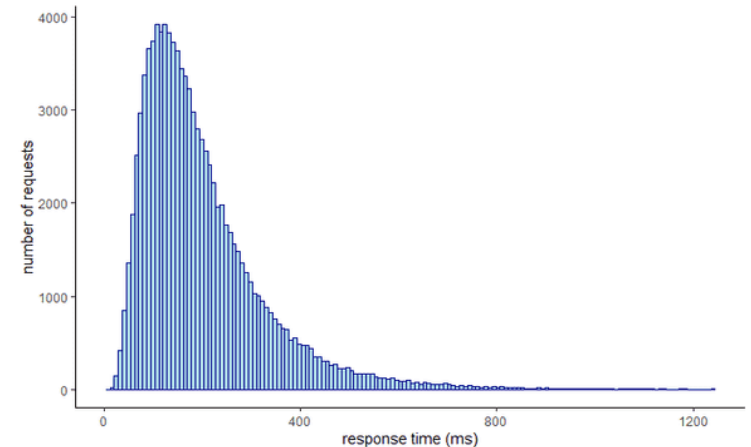
Programs written in this functional style are automatically parallelized and executed on a large cluster of commodity machines. The run-time system takes care of the details of partitioning the input data, scheduling the pro-

grams, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computations we were trying to perform but hides the messy details of parallelization, fault-tolerance, data distribution and load balancing in a library. Our abstraction is in-

Response times are critical! DID YOU SEE THIS!??!

- If a response time is not within *40 milliseconds*
 - A user will LEAVE!
 - Impacts google rankings!
 - There are delays in data transfer (latencies per request)
 - Did you love measuring these?! 😊
- Fun fact is that there is a VERY LONG TAIL graph
 - Slow responses DO happen
 - So how can we mitigate those?!



Throughput

- Typically reported as ***requests per second***
 - Note: Latency is for a single request
 - Throughput is a bulk measure of **many** requests
 - You will love measuring this!
- When you have millions of requests per second *and* bursts
 - We need to split the load across multiple nodes!
- “Scale-out”
 - Add more nodes to increase throughput...
 - Does it always work?

Availability!

- 24/7
 - 99%, 99.999%, 99.9999%...

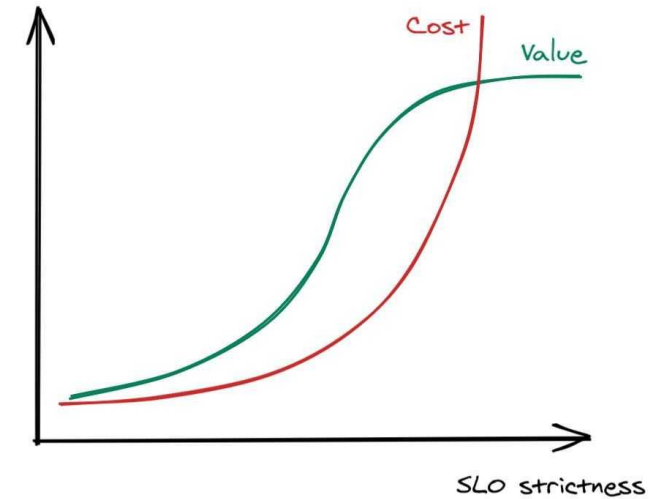
Availability %	Downtime per year ^[note 1]	Downtime per quarter	Downtime per month	Downtime per week	Downtime per day (24 hours)
90% ("one nine")	36.53 days	9.13 days	73.05 hours	16.80 hours	2.40 hours
95% ("one and a half nines")	18.26 days	4.56 days	36.53 hours	8.40 hours	1.20 hours
97%	10.96 days	2.74 days	21.92 hours	5.04 hours	43.20 minutes
98%	7.31 days	43.86 hours	14.61 hours	3.36 hours	28.80 minutes
99% ("two nines")	3.65 days	21.9 hours	7.31 hours	1.68 hours	14.40 minutes
99.5% ("two and a half nines")	1.83 days	10.98 hours	3.65 hours	50.40 minutes	7.20 minutes
99.8%	17.53 hours	4.38 hours	87.66 minutes	20.16 minutes	2.88 minutes
99.9% ("three nines")	8.77 hours	2.19 hours	43.83 minutes	10.08 minutes	1.44 minutes
99.95% ("three and a half nines")	4.38 hours	65.7 minutes	21.92 minutes	5.04 minutes	43.20 seconds
99.99% ("four nines")	52.60 minutes	13.15 minutes	4.38 minutes	1.01 minutes	8.64 seconds
99.995% ("four and a half nines")	26.30 minutes	6.57 minutes	2.19 minutes	30.24 seconds	4.32 seconds
99.999% ("five nines")	5.26 minutes	1.31 minutes	26.30 seconds	6.05 seconds	864.00 milliseconds
99.9999% ("six nines")	31.56 seconds	7.89 seconds	2.63 seconds	604.80 milliseconds	86.40 milliseconds
99.99999% ("seven nines")	3.16 seconds	0.79 seconds	262.98 milliseconds	60.48 milliseconds	8.64 milliseconds
99.999999% ("eight nines")	315.58 milliseconds	78.89 milliseconds	26.30 milliseconds	6.05 milliseconds	864.00 microseconds
99.9999999% ("nine nines")	31.56 milliseconds	7.89 milliseconds	2.63 milliseconds	604.80 microseconds	86.40 microseconds

- Don't want a single point of failure (SPoF)
 - So we need to make sure we have replicas/redundancy!
- When you have 1000s of components, some will fail!
 - Need to incorporate **FAILURE** into our designs

https://en.wikipedia.org/wiki/High_availability

It is all about the *Tradeoffs*!

- Throughput versus cost
 - More hardware means more \$\$
 - When is it worth it? When isn't it?
- Performance versus availability
 - Fast but components may fail!
- ***Need to be able to measure and discuss the tradeoffs in the work you do in this course!***
- This all depends on the context of the requirements of the system you are building, there is no ONE right answer!



<https://robertovitillo.com/why-you-should-measure-tail-latencies/>

Networking Concepts!

- Sufficient Networking for this course
 - Network Communications Basics
 - Remote Procedure Calls
 - Distributed Coordination
 - Time in distributed systems
 - Virtual Machines and Containers

Networking is so COOL! requires a whole course!

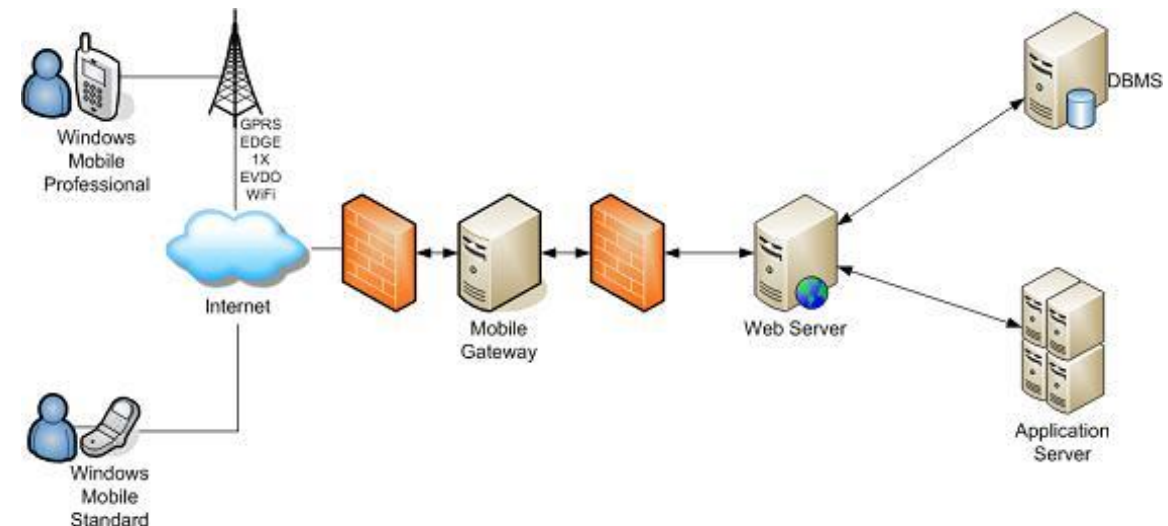
Network Communications Basics

- Scaling-Out means...
 - Software runs on multiple machines
 - Databases store data on multiple storage nodes
 - WOAH!!!!
- Scalable software solutions
 - Are distributed across multiple nodes in multiple locations
 - Each node processes events **concurrently**
 - Nodes exchanging messages over a network with **non-zero** transmission time
 - Messages are **unreliable**, they might be lost
 - Messages can (will?!) be intercepted by “bad” people
- A Distributed System **must** handle all these problems gracefully
 - Otherwise, your on-call schedule will be a nightmare ☹️

Mobile Banking Example

Every distributed system communicates over a network

- A cell phone app sends a request over the **cellular network** addressed to a bank to retrieve bank balance
 - Request is routed across the **internet to the bank's web server**
 - Bank's **web server** authenticates the request (checks it's you) and sends a request to a database server for balance.
 - **Database** server reads bank balance from disk and returns it to the **web server**
 - Web server sends balance in a reply message to the app, which is routed over the internet and the cellular network
 - Balance appears on your mobile device!
- NOW have 5 million users...
MAGIC!!!



Some Network Acronyms

- **LAN** Local Area Network. Your House. Private.
- **Wireless LAN** LAN over WiFi.
- **WAN** Wide Area Network.
Assembly of LANs, e.g. a university
- **The Internet** Public assembly of WANs
- **VPN** Virtual Private Network.
Private network over a public net.
 - requires cryptography

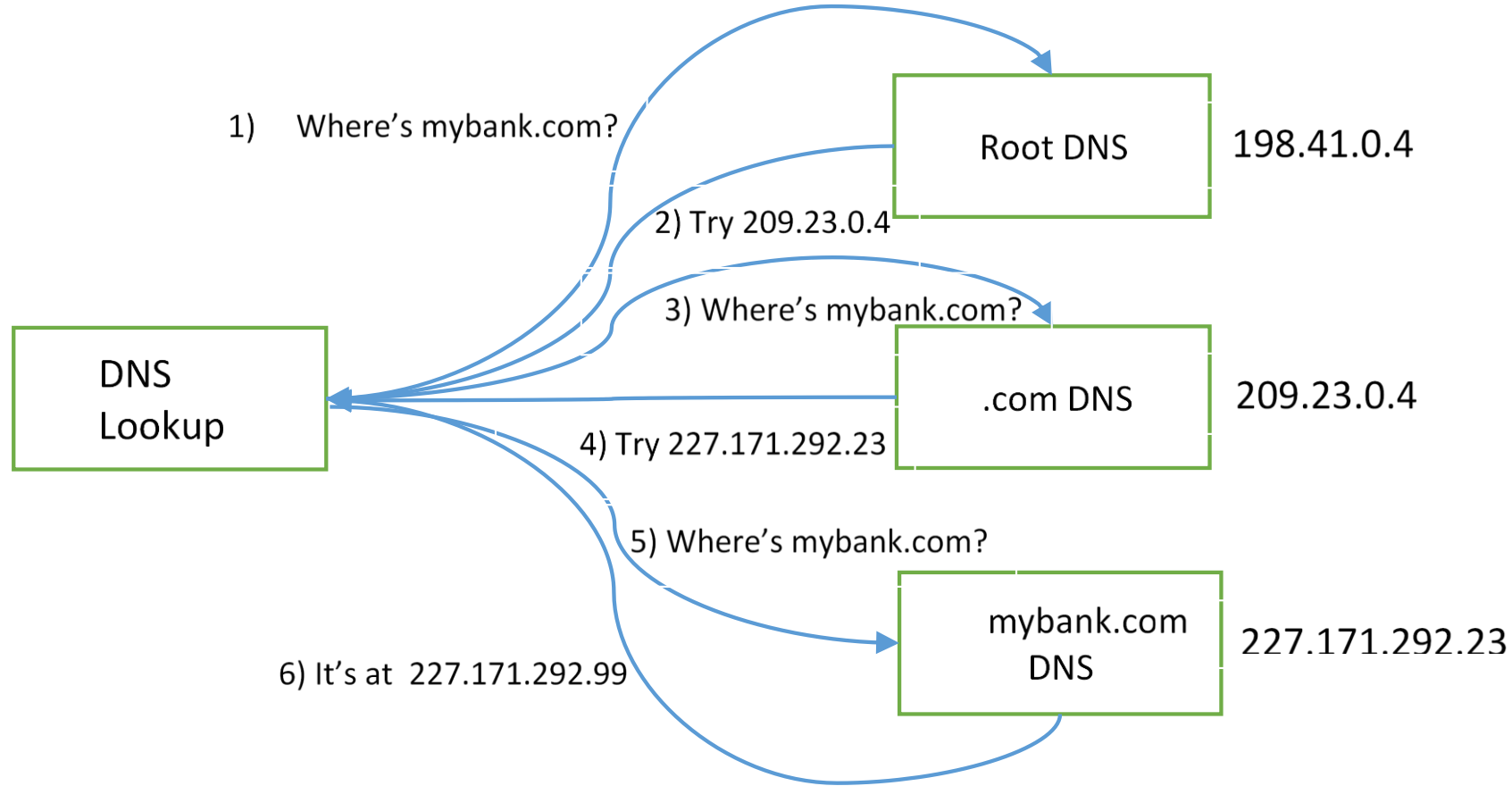
Current Protocols

- Network Protocols since 1995 are all “TCP/IP stacks”
 - Transmission Control Protocol/Internet Protocol
- Software systems on the internet communicate using the Internet Protocol suite
 - Host addressing
 - Data transmission formats
 - Message routing and delivery characteristics

Network Layer: Internet Protocol (IP)

- IP defines:
 - how hosts are assigned addresses on the internet, and
 - how messages transmitted between two hosts who know each other's addresses
- Every device has its own Internet Protocol (IP) address.
- The location of an IP address stored in a directory service known as **Domain Naming Service (DNS)**
- **DNS** is a widely distributed, **hierarchical database**
- The address book of the internet!

Network: DNS (Domain Naming System) Lookup



IP Addresses – Simple

IP Numbers

- The most visible part of networking on all networks
 - The Internet, Your Home LAN, Campus-wide WANs
- Two versions
 - IPv4
 - 32-bit addresses, written as 97.113.122.65
 - No longer enough addresses for the whole world
 - Still the most common
 - IPv6
 - 128-bit addresses
 - Will “always” have sufficient numbers
 - Slow adoption
 - Ignored in this course

Public and Private IP Addresses

- IP addresses on The Internet must be unique
 - No two machines (NICs) can have the same address
- Some subranges cannot be allocated on The Internet
 - The subranges only to be used within **private** networks
 - 192.168.0.0/16 (CIDR format... Classless Inter-Domain Routing)
 - 192.168.0.0 through 192.168.255.255
 - Also written 192.168.x.x
 - 10.0.0.0/8
 - 10.0.0.0 through 10.255.255.255
 - Also written 10.x.x.x
- **CIDR Format** – the number *after* the slash is the number of **fixed** bits. Every 8 bits is one of the four parts of the full IP number

Public and Private Networks

- Public

- The Internet. Every IP address must be unique

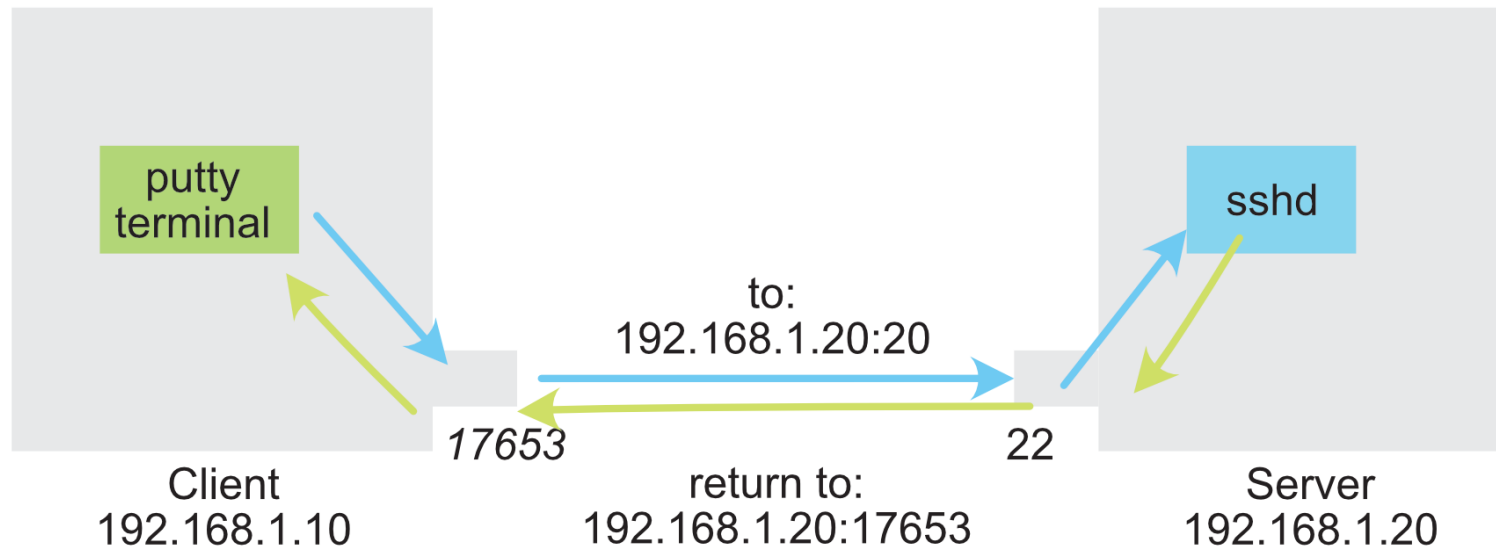
- Private

- Your home
 - Northeastern
 - A corporate LAN
 - Many companies have multiple separate address spaces 'cos they ran out
 - Use 192.168.x.x and 10.x.x.x

Addresses and Ports

- The IP address identifies the machine
 - To be precise, it identifies the Network Interface Card
 - Servers have multiple NICs, laptops and PCs only have one
- The **port** identifies the (software) service within that machine
- “Well known” services have defined port numbers
 - **22** for ssh
 - **80** for http
 - **443** for https
 - **8080** is used a lot as default for http, so that it doesn't accidentally block the local web server on port 80

Message Exchange using IP Addresses



1. Client asks for a socket to send a message, operating system creates new port number *17653*
2. Client sends to 192.168.1.20 on port 22
3. OS on server receives messages, looks up socket ownership table and routes message to sshd
4. Sshd does its work, then replies
5. Server sends to 192.168.1.10 on port 17653
6. Client OS looks up socket ownership table and routes the message to putty (windows) or terminal (macOS)

Domain Naming System (DNS)

- A global hash of names to IP addresses
 - Use `nslookup`
 - `northeastern.edu` is `155.33.17.68`
 - DNS is a highly reliable, globally distributed database
- This is what it means to “own a domain”

Firewalls

- Sits between the OS and the network
- Only allows traffic in and out on defined ports
 - Conceptually very simple
- AWS Security Groups are firewalls

Cryptography

Some Uses of Cryptography

- Protect sensitive information as it travels over insecure networks
 - Intermediate hosts might have been hacked
 - Encrypt on the client, decrypt at the server
- Protect sensitive information when it is stored
 - Encrypted drives. Protect against laptop theft
- Create logical networks – VPN
 - Several VLANs or VPNs share the same physical network
 - Different crypto keys for each virtual network – cannot decode traffic on other network

Very, very quick intro to Cryptography

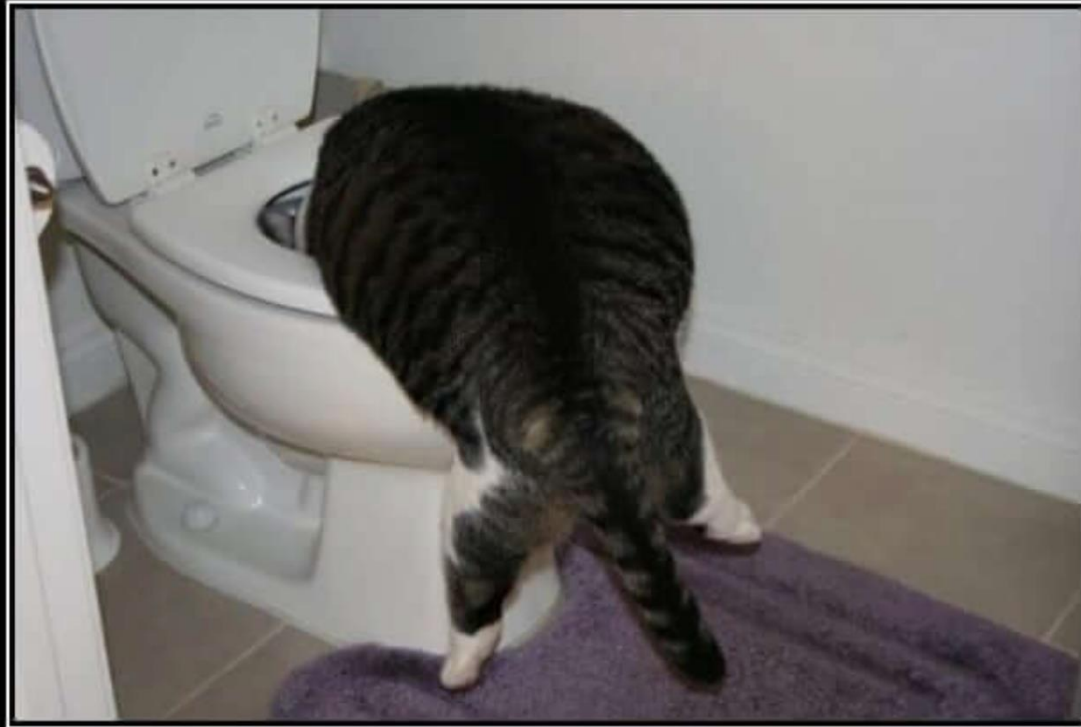
- All current cryptographic technologies rely on secret keys
 - A key is a long sequence of bits
- **Symmetric keys** Both nodes have the same key
 - Problem – how does **Alice** safely send the key to her friend **Bob**?
 - Fast to encode and decode
- **Asymmetric keys, Public Key Encryption PKI**
 - SSH uses **PKI**
 - PKI used to *exchange a symmetric key*
- https (port 443) uses PKI, http (port 80) does not

HTTP API

- HTTP is simple in practice
 - Web servers are common
 - Uses an existing protocol stack
 - DNS
 - Corporate firewalls already configured to pass http and https
 - Language Independent (Not just Java or Microsoft C#)
 - API is loosely coupled
 - Changing the API DOES NOT require recompiling all clients

HTTP Cats!

403 Forbidden



403
Forbidden

Important HTTP Response Codes

- 200 OK – everything is Okay
- 202 Created – More specifically, the POST or PUT succeeded
- 400 Bad Request – Bad data from the client
- 403 Forbidden – Security error, client probably needs to log in
- 404 Not Found – URL does not resolve to anything
- 500 Internal Server Error – The default error, uncaught exception
- 502 Bad Gateway – A server behind this one failed. Load balancers often return 502
- 503 Service Unavailable – This service is dead Jim

Infrastructure as Code

Terraform

Automation vs Manual

- Manually clicking to create VMs and Networks is
 - Slow
 - Boring
 - Error Prone
 - Not Repeatable ☹️
- Automation solves all those problems 😊
- Infrastructure as Code

```
provider "aws" {  
  region = "us-west-2"  
}  
  
data "aws_ami" "ubuntu" {  
  most_recent = true  
  
  filter {  
    name = "name"  
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]  
  }  
  
  owners = ["099720109477"] # Canonical  
}  
  
resource "aws_instance" "app_server" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "learn-terraform"  
  }  
}
```

Advantages of Configuration Files

- Repeatable
- Can be stored in Git
 - Reviewable by other people
 - History
 - Knowledge is retained in the company
 - Safe to use at 3am!

Various IaC Tools

- Terraform
 - OpenTofu open source
 - We will use Terraform in our course
- Puppet
- Chef
- Ansible
- Kubernetes

Readings for Infrastructure as Code

- [https://en.wikipedia.org/wiki/Infrastructure as code](https://en.wikipedia.org/wiki/Infrastructure_as_code)
- <https://developer.hashicorp.com/terraform/tutorials/aws-get-started>